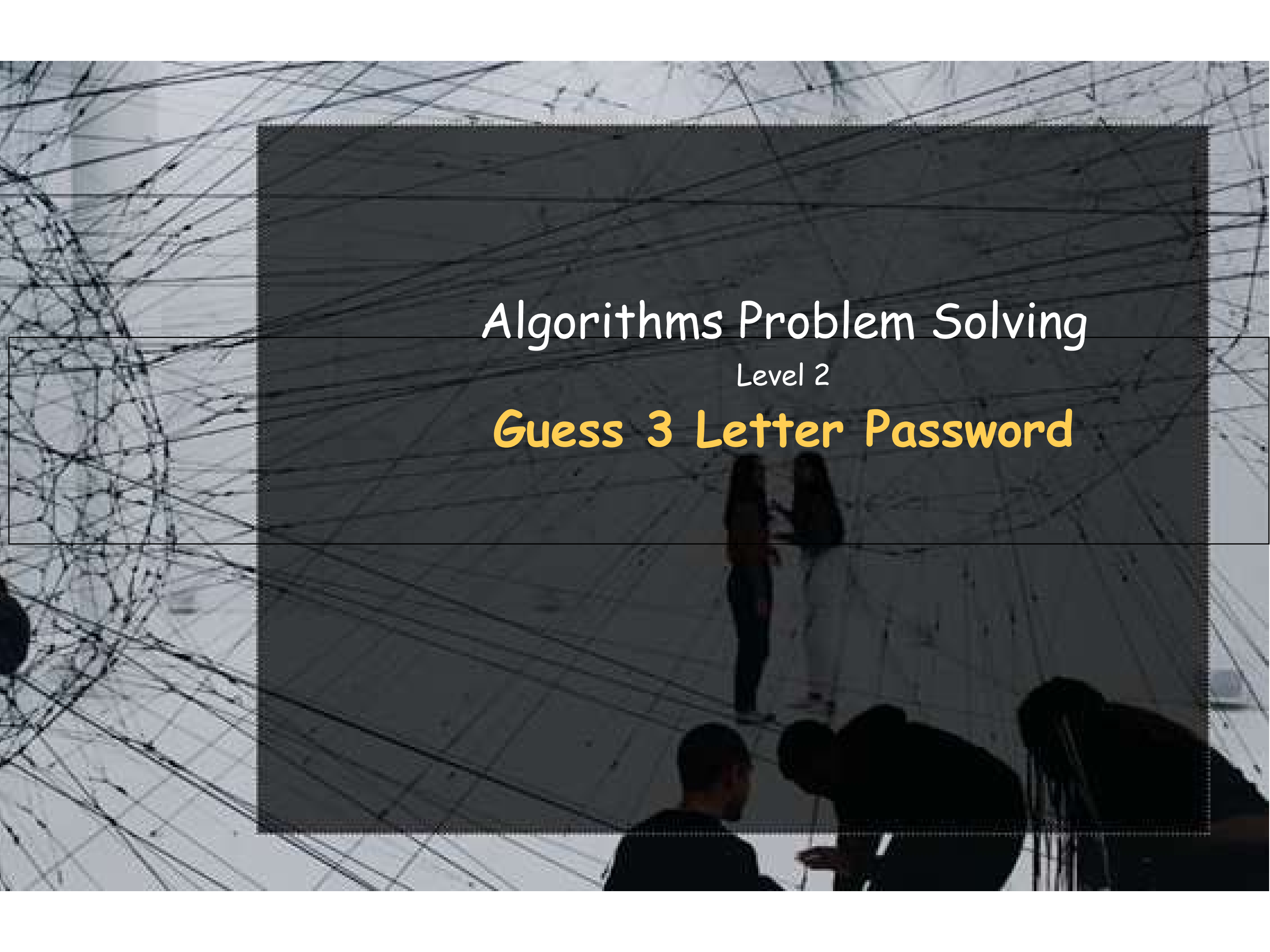


Algorithms Problem Solving

Level 2

Guess 3 Letter Password

The background of the slide is a photograph of a meeting room. A large screen at the front of the room displays a complex network diagram with many nodes and connecting lines. In the foreground, the silhouettes of several people are visible, some standing and some sitting, appearing to be in a discussion or presentation. The overall lighting is dim, with the screen being the primary light source.

Algorithms Problem Solving

Level 2

Guess 3 Letter Password

Definition

- ◆ A program that tries to guess a 3-letter password containing uppercase letters from `A` to `Z`.
- ◆ The program generates every possible combination until it finds the password entered by the user.

Explanation

- ◆ ``ReadPassword()`` reads the original password from the user and returns it as a ``string``.
- ◆ ``FindPassword()`` uses three nested ``for`` loops to generate all possible 3-letter combinations.
- ◆ ``counter`` counts the number of attempts.
- ◆ ``word`` is a ``string`` used to build the current 3-letter combination.
- ◆ ``push_back()`` adds each generated character to the end of the string.
- ◆ ``word == OriginalPassword`` compares the generated combination with the entered password.
- ◆ When the password is found, the program prints the password and number of attempts, then uses ``return`` to stop the function.

■ Steps in main()

- 1 Read the 3-letter password from the user.
- 2 Start the first loop from `A` to `Z`.
- 3 Start the second loop from `A` to `Z`.
- 4 Start the third loop from `A` to `Z`.
- 5 Increase `counter` by 1 for every generated combination.
- 6 Create `word` and add `i`, `j`, and `k` using `push\_back()`.
- 7 Print the current attempt.
- 8 Compare `word` with the original password.
- 9 If they are equal, print the result and stop using `return`.
- 10 If all combinations are checked without finding the password, print `Password not found`.

💡 Example

◆ Input:

AAF

◆ The program starts generating combinations:

Trail [1]: AAA

Trail [2]: AAB

Trail [3]: AAC

Trail [4]: AAD

Trail [5]: AAE

Trail [6]: AAF

◆ At Trail 6, `word` is equal to `OriginalPassword`.

➡ Result

◆ The program prints:

The password is AAF

Found after 6 Trail(s)

◆ The password `AAF` was found after 6 attempts.

✚ Note

- ◆ `push_back()` adds one character at a time to the string.
- ◆ For example:
 - `word.push_back(i);`
 - `word.push_back(j);`
 - `word.push_back(k);`
- ◆ If `i = A`, `j = A`, and `k = F`, the resulting `word` is `AAF`.
- ◆ There are $26 \times 26 \times 26 = 17,576$ possible 3-letter combinations.
- ◆ The program checks the combinations in alphabetical order, starting from `AAA` and ending at `ZZZ`.

⊘ Warning

- ◆ The program does not validate that the entered password contains exactly 3 uppercase letters.
- ◆ For example, the user could enter `AB`, `abcd`, or `123`, and the program would still accept the input.
- ◆ The word `Trail` in the output should technically be `Trial`, meaning an attempt.

Comparison

- ▣ ``counter`` → counts how many attempts have been made.
- ▣ ``word`` → stores the current generated password.
- ▣ ``OriginalPassword`` → stores the password entered by the user.
- ▣ ``word == OriginalPassword`` → checks whether the current attempt matches the password.

✦ Summary

- ✓ The program guesses a 3-letter uppercase password by generating combinations.
- ✓ Three nested loops generate all possible combinations from `AAA` to `ZZZ`.
- ✓ `counter` counts the number of attempts.
- ✓ `push\_back()` builds the generated password character by character.
- ✓ `==` checks whether the generated password matches the original password.
- ✓ `return` stops the search immediately when the password is found.
- ✓ Maximum number of attempts = `17,576`.

Thank you 😊